Day 2 – AWS IAM vs Azure RBAC

-Samarth Ballal

Introduction :

For Day 2, I focused on understanding how access to cloud resources is managed in AWS and Azure. The main topic was a comparison between AWS Identity and Access Management (IAM) and Azure Role-Based Access Control (RBAC). Before doing this comparison, I understood that access management is an important part of cloud security because not every user or service should have access to everything. While both AWS IAM and Azure RBAC are designed to control access, they approach permissions in slightly different ways. Looking at both side by side helped me understand that the cloud platforms may use different terminology and methods, but the security goal is largely the same: give the right access to the right person or service and avoid unnecessary permissions.

AWS IAM – What I Understood :

AWS IAM is used to manage access to AWS services and resources. It works with users, groups, roles, and policies. Permissions are normally described using JSON-based policies, which specify what actions are allowed or denied and which resources those permissions apply to. One part that stood out to me was the use of IAM roles. Roles can provide temporary permissions to AWS services such as EC2, which means credentials do not have to be hardcoded on the server. This makes the setup more secure and is useful when an AWS service needs to interact with another AWS resource.

Azure RBAC – What I Understood :

Azure RBAC is Microsoft's authorization system for controlling access to Azure resources. Instead of using JSON policies in the same way as AWS IAM, Azure uses built-in and custom roles. These roles can be assigned at different levels, such as a subscription, resource group, or individual resource. Azure also works closely with Microsoft Entra ID for identity management and authentication. This helped me see the difference between identifying a user and deciding what that user is allowed to do. RBAC mainly focuses on assigning the appropriate permissions through roles and scopes.

AWS IAM vs Azure RBAC – Feature Comparison :

Feature	AWS IAM	Azure RBAC
Identity Service	IAM	Microsoft Entra ID
Access Control Method	JSON-based policies	Built-in and custom roles
Permission Assignment	Users, Groups, Roles	Users, Groups, Service Principals
Scope	AWS resources	Subscription, Resource Group, Resource
Temporary Access	IAM Roles	Managed Identities & Role Assignments
Least Privilege Support	Yes	Yes

Similarities I Noticed :

After comparing the two systems, I noticed that both follow the principle of least privilege. This means a user, application, or service should receive only the permissions needed to complete its task. Both platforms also support role-based access, custom permission options, auditing, and secure access management. This similarity is important because access control is not only about giving permissions; it is also about reducing the chances of accidental

or unauthorized changes. Keeping permissions limited makes the overall cloud environment easier to manage and more secure.

Key Differences I Observed :

The main difference I noticed is how permissions are represented and assigned. In AWS, permissions are defined through policies, commonly written in JSON, and these policies can be attached to users, groups, or roles. In Azure, access is organized around role assignments, with roles being applied at different scopes. Another difference is the way temporary access is handled. AWS commonly uses IAM roles, while Azure uses managed identities and role assignments. Although the implementation is different, both approaches help avoid unnecessary long-term credentials.

Simple Practical Example :

To make the comparison easier to understand, I thought of a simple situation where a cloud application needs to access stored files. In AWS, the application could use an IAM role with a policy that provides only the required S3 permissions. In Azure, a similar requirement could be handled by assigning an appropriate RBAC role to the identity at the required scope.

The important point is not just which cloud platform is being used. The important part is making sure the application receives only the access it actually needs. This example helped me connect the theoretical comparison with the practical access-control work from Day 1.

What I Learned Today :

The biggest takeaway for me from Day 2 was that access management is a fundamental part of cloud security. I had already worked with IAM during the previous day's AWS setup, but comparing it with Azure RBAC gave me a better understanding of why permissions need to be planned carefully.

I also understood that different cloud providers may implement access control differently, but the underlying security principle remains similar. Rather than giving broad access, permissions should be kept specific and limited to what is required.

Day 2 Takeaway :

Overall, Day 2 helped me understand the difference between AWS IAM and Azure RBAC in a more practical way. AWS IAM is centered around users, groups, roles, and JSON-based policies, while Azure RBAC uses built-in or custom roles that can be assigned at different scopes. Both provide mechanisms for following least privilege and managing access securely.

Comparing the two made the topic easier for me to remember because I could see both the similarities and the differences instead of studying each service separately.